

Answer Format

1. Identify the data structure
2. Justification (2-3 lines)
3. Structure definition
4. Function's to be included in code
5. C program

Question-1

A multiplex theater has a single booking counter where customers arrive one after another to reserve movie tickets. Each customer specifies the number of tickets required, and their requests are processed in the order of arrival. The counter can handle only a limited number of booking requests at a time. As each booking is completed, he/she should get out of booking counter. The system must maintain a count of total bookings successfully processed, and at the end of the session, if the number of completed bookings is greater than 10, it should display “Bravo, Too Good Performance”, otherwise it should display “Poor Performance”. Write a modular C program to simulate this booking system with all the required functionalities.

C Code

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5    // maximum capacity of booking counter

// Structure for Queue
struct Queue {
    int front, rear;
    int data[MAX];
};

// Initialize queue
void init(struct Queue *q) {
    q->front = -1;
    q->rear = -1;
}
```

// Check if queue is empty

```
int isEmpty(struct Queue *q) {  
    if (q->front == -1 || q->front > q->rear)  
        return 1;  
    else  
        return 0;  
}
```

// Check if queue is full

```
int isFull(struct Queue *q) {  
    if (q->rear == MAX - 1)  
        return 1;  
    else  
        return 0;  
}
```

// Add customer request (enqueue)

```
void enqueue(struct Queue *q, int tickets) {  
    if (isFull(q)) {  
        printf("Counter is full! Customer must wait.\n");  
        return;  
    }  
  
    if (q->front == -1)  
        q->front = 0;  
  
    q->rear++;  
    q->data[q->rear] = tickets;  
  
    printf("Customer added with %d tickets\n", tickets);  
}
```

// Remove customer request (dequeue)

```
int dequeue(struct Queue *q) {  
    int value;
```

```

    if (isEmpty(q)) {
        printf("No bookings to process\n");
        return -1;
    }

    value = q->data[q->front]; // take front element
    q->front++;                // move front forward

    // reset queue if all elements are removed
    if (q->front > q->rear) {
        q->front = q->rear = -1;
    }

    printf("Processed booking: %d tickets\n", value);

    return value;
}

// Display queue elements
void display(struct Queue *q) {
    int i;

    if (isEmpty(q)) {
        printf("No waiting customers\n");
        return;
    }

    printf("Waiting customers: ");

    for (i = q->front; i <= q->rear; i++) {
        printf("%d ", q->data[i]);
    }

    printf("\n");
}

```

```

// Process all bookings and calculate result
void processAll(struct Queue *q) {
    int count = 0;
    int value;

    printf("\nProcessing all bookings...\n");

    // SIMPLE VERSION OF LOOP (EASY TO UNDERSTAND)
    while (!isEmpty(q)) {

        // Step 1: remove front customer
        value = dequeue(q);

        // Step 2: if valid booking, increase count
        if (value != -1) {
            count = count + 1;
        }
    }

    // Final result
    printf("\nTotal bookings processed: %d\n", count);

    // Performance check
    if (count > 10) {
        printf("Bravo, Too Good Performance\n");
    } else {
        printf("Poor Performance\n");
    }
}

// Main function
int main() {
    struct Queue q;
    init(&q);

    int choice, tickets;

```

```
while (1) {
    printf("\n--- Multiplex Booking System ---\n");
    printf("1. Add Customer Request\n");
    printf("2. Process One Booking\n");
    printf("3. Display Waiting List\n");
    printf("4. Process All Bookings\n");
    printf("5. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {

        case 1:
            printf("Enter number of tickets: ");
            scanf("%d", &tickets);
            enqueue(&q, tickets);
            break;
        case 2:
            dequeue(&q);
            break;
        case 3:
            display(&q);
            break;
        case 4:
            processAll(&q);
            break;
        case 5:
            exit(0);
        default:
            printf("Invalid choice\n");
    }
}
return 0;
}
```

Question -2

A smart vending counter in a supermarket accepts only specific token values of 10, 20, and 30 from customers. Whenever a customer inserts a token, the system stores it in the order it is received and processes the tokens one by one. If any token other than 10, 20, or 30 is inserted, the system immediately rejects it and displays an “Invalid Token” message. After processing all valid tokens, the system calculates the total amount collected and dispenses products based on the final sum as follows: if the total is 10, “Juice” is dispensed; if 20, “Milk”; if 30, “Sugar”; if 40, “Fruits”; if 50, “Veggies”; and if 60, “Oats”. Write a modular C program to simulate this vending counter system with operations to accept tokens, process them in the same order they were entered, display invalid inputs when necessary, calculate the total amount, and print the corresponding product based on the final sum.

C Code

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 10    // maximum tokens stored

// Queue structure
struct Queue {
    int front, rear;
    int data[MAX];
};

// Initialize queue
void init(struct Queue *q) {
    q->front = -1;
    q->rear = -1;
}
```

// Check empty

```
int isEmpty(struct Queue *q) {  
    if (q->front == -1 || q->front > q->rear)  
        return 1;  
    else  
        return 0;  
}
```

// Check full

```
int isFull(struct Queue *q) {  
    if (q->rear == MAX - 1)  
        return 1;  
    else  
        return 0;  
}
```

// Insert token

```
void enqueue(struct Queue *q, int token) {  
    if (isFull(q)) {  
        printf("Counter full! Cannot accept more  
tokens\n");  
        return;  
    }  
  
    if (q->front == -1)  
        q->front = 0;
```

```
q->rear++;  
q->data[q->rear] = token;  
printf("Token %d accepted\n", token);  
}
```

// Remove token

```
int dequeue(struct Queue *q) {  
    int value;  
    if (isEmpty(q)) {  
        return -1;  
    }  
    value = q->data[q->front];  
    q->front++;  
  
    if (q->front > q->rear) {  
        q->front = q->rear = -1;  
    }  
    return value;  
}
```

// Display product based on total sum

```
void displayProduct(int total) {  
    printf("\nTotal Amount = %d\n", total);  
    if (total == 10)  
        printf("Product Dispensed: Juice\n");  
}
```



```
else if (total == 20)
    printf("Product Dispensed: Milk\n");
else if (total == 30)
    printf("Product Dispensed: Sugar\n");
else if (total == 40)
    printf("Product Dispensed: Fruits\n");
else if (total == 50)
    printf("Product Dispensed: Veggies\n");
else if (total == 60)
    printf("Product Dispensed: Oats\n");
else
    printf("No product available for this total\n");
}
```

// Process tokens (count + sum)

```
void processTokens(struct Queue *q) {
    int value;
    int total = 0;
    int count = 0;
    printf("\nProcessing tokens...\n");
```

// PROCESS ONE BY ONE USING DEQUEUE

```
while (!isEmpty(q)) {
    value = dequeue(q);
    // valid token check
    if (value == 10 || value == 20 || value == 30) {
```

```
        total = total + value; // sum calculation
        count = count + 1;      // count tokens
    }
}

// Output results
printf("\nTotal Tokens Processed = %d\n", count);
displayProduct(total);
}
```

// Main function

```
int main() {
    struct Queue q;
    init(&q);
    int choice, token;
    while (1) {

        printf("\n--- Smart Vending Counter ---\n");
        printf("1. Insert Token\n");
        printf("2. Process Tokens\n");
        printf("3. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
```

```
        case 1:
            printf("Enter token (10, 20, 30 only): ");
            scanf("%d", &token);
            if (token == 10 || token == 20 || token ==
30) {
                enqueue(&q, token);
            } else {
                printf("Invalid Token Rejected!\n");
            }
            break;
        case 2:
            processTokens(&q);
            break;
        case 3:
            exit(0);
        default:
            printf("Invalid choice\n");
    }
}
return 0;
}
```